
BY COMMUNITY • FOR COMMUNITY •
#GLOBALAZURE
TORINO



Azure Meetup
TORINO

alveo®
EXPERTISE

TD SYNNEX

#GlobalAzureTorino

Understanding CALCULATE The Queen of all DAX Functions

Prof. Francesco Bergamaschi, M.Eng, M.Econ, MBA
Prof. Lorenzo Bergamaschi, M.DAX

Azure Meetup TORINO

alveo EXPERTISE

TD SYNNEX

About me

- Master of Science in IT Engineering, Master of Science in Economics, MBA
- Professor of BI & Analytics @UniBO & @Bologna Business School
- Visiting Professor @UNIVPM e @UNICATT
- BI & Analytics Consultant
- Lecturer in DAX and Tabular since 2014
- Co-founding member of kubisco (www.kubisco.com)
- Co-founding member of the Power BI User Group Italy
- Current Focus: Composite Models and Visual Context



kubisco
www.kubisco.com

Agenda

Topic 1 The Overall Scenario

- Introducing CALCULATE
- Why modifying the Filter Context?
- What modifications can be done to the Filter Context through CALCULATE?

Topic 2 Filters are Tables

- Filter and Filter Context Definition
- More on Filters
- Filter Modifier

Topic 3 Global Modifiers and Algorithm

- CALCULATE Context Transition and Global Modifiers
- CALCULATE Algorithm
- CALCULATE Alogrithm re-cap examples (Demo)

#GlobalAzureTorino

Topic 1 The Overall Scenario

Introducing CALCULATE
Why modifying the Filter Context?
What modifications can be done to the Filter Context through CALCULATE?

Azure Meetup TORINO
alveo EXPERTISE
TD SYNNEX

#GlobalAzureTorino

Introducing CALCULATE



CALCULATE is preceived as a complex function. DAX is perceived as a complex language. Both are misperceptions.

CALCULATE is a **simple** and **powerful** function. DAX is a **simple** and **powerful** language. Point is that **simple** does not mean **easy**.

CALCULATE and DAX in general can be used in complex scenarios as they are powerful but you can also use both in simple scenarios and complexity does not show up.

It is what we want to do that MIGHT be complex, not CALCULATE or DAX in general. As they are both powerful, theyt can ALSO be used in COMPLEX scenarios.

#GlobalAzureTorino

Introducing CALCULATE

DAX Pillars are only six (learn them)!

Filter Context
Row Context
Iterators
Context Transition
Expanded Tables
Visual Context

#GlobalAzureTorino

Introducing CALCULATE

CALCULATE documentation (dax.guide)

CALCULATE (<Scalar Expression>, [Filter/GlobalModifier 1], [Filter/GlobalModifier 2], …)

In short, CALCULATE evaluates a **scalar** DAX expression in a Filter Context modified by Filters and/or Modifiers.

CALCULATE works ONLY with the Filter Context. The **scalar** expression must, therefore, be meaningful in absence of Row Context

#GlobalAzureTorino

Introducing CALCULATE



CALCULATE works ONLY with the Filter Context. The **scalar** expression must, therefore, **be meaningful in absence of Row Context**

CALCULATE ([Measure], …) OK

CALCULATE (< Explicit scalar DAX code >, …) OK

CALCULATE (Table[Column], ….) NOT OK (Row Context needed)

CALCULATE (RELATED (Table[Column], …) NOT OK (Row Context needed)

#GlobalAzureTorino

Introducing CALCULATE

CALCULATE is implicitely called anytime a reference to a measure is done.

[Measure] is executed as CALCULATE ([Measure])

Due to the Context Transition (more on this later) performed by CALCULATE, the practice of referencing measures omitting the table name has been put in force **([Measure] and not Table[Measure]**). In fact, Measures can be placed in any table without affecting their results. They have nothing to do with any particular table.

Columns, on the contrary, are hardwired to tables, so it makes sense to reference them as **Table[Column]**

#GlobalAzureTorino

Introducing CALCULATE

CALCULATETABLE semantic is identical to CALCULATE, the only difference being:

CALCULATE evaluates a scalar DAX expression in a Filter Context modified by Filters and/or Global Modifiers,

while

CALCULATETABLE evaluates a table DAX expression in a Filter Context modified by Filters and/or Global Modifiers.

We shall, for brevity, describe CALCULATE only

#GlobalAzureTorino

Why modifying the Filter Context?



CALCULATE (<Scalar Expression>, [Filter/GlobalModifier 1], [Filter/GlobalModifier 2], …)

In short, CALCULATE evaluates a DAX expression in a Filter Context modified by Filters and/or Global Modifiers.

Why modifying the Filter Context? Many reasons:
1 – to avoid changing the internals of measures when creating variants;

2 – to avoid getting more than what is needed from the Filter Context and selecting what is needed *ex-post*;

3 – to simplify and shorten the DAX code;

4 – to simplify the DAX code maintenance.

#GlobalAzureTorino

Why modifying the Filter Context?

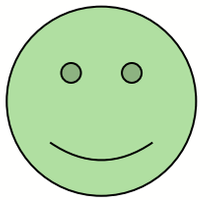
```
Sales =
SUMX(
    Sales,
    Sales[OrderQuantity]*Sales[UnitPrice]
)
```

```
Sales M Customers =
SUMX(
    FILTER (
        Sales,
        RELATED(Customer[Gender])="M"
    ),
    Sales[OrderQuantity]*Sales[UnitPrice]
)
```



[Sales] is our base measure, how to generate two variants of it?

```
Sales M Customers CALCULATE =
CALCULATE(
    [Sales],
    Customer[Gender] = "M"
)
```



#GlobalAzureTorino

Why modifying the Filter Context?




```
1 Sales M Customers CALCULATE =
2 CALCULATE(
3     [Sales],
4     Customer[Gender] = "M"
5 )
```

Here a Filter on Customer[Gender] has been applied to the Filter Context, so:
1 – We are changing the Filter Context, so we do NOT need to change the measure internals and we can reference it;
2 – We are, therefore, getting only what we need from the Filter Context (only “M”);
3 – the DAX code is short and easy to read, understand and maintain.

#GlobalAzureTorino

Why modifying the Filter Context?



Time Intelligence Calculations;

Ratio Calculations;

Advanced Calculations in general.

All of the above need filter context manipulation in order to create neat reports.

#GlobalAzureTorino

What modifications can be done to the Filter Context through CALCULATE?



CALCULATE (<Scalar Expression>, [Filter/GlobalModifier 1], [Filter/GlobalModifier 2], …)

1 – Adding Filters (explicitly/implicitly, overwriting/intersecting existing ones);

2 – Removing Filters;

3 – Modify the Columns involved in the model Relationships;

4 – Modify the Cross-Filter direction in the model Relationships.

#GlobalAzureTorino

What modifications can be done to the Filter Context through CALCULATE?

	Explicit Filter Addition	Implicit Filter Addition (Context Transition)
Overriding Existing Filters	CALCULATE ([Measure], Table[Column] = Value)	SUMX (VALUES (Product[Color]), [TopPriceListProductSales])
Intersecting Existing Filters	CALCULATE ([Measure], KEEPFILTERS (Table[Column] = Value))	SUMX (KEEPFILTERS (VALUES (Product[Color])), [TopPriceListProductSales])

Implicit Filter Addition happens through Context Transition that needs two ingredients to be triggered: (1) at least one row context active and (2) a CALCULATE statement. But where is the CALCULATE statement in the SUMX expression? **Remember that [Measure] is CALCULATE [Measure]!**

#GlobalAzureTorino

Topic 2 Filters are Tables

Filter and Filter Context Definition
More on Filters
Filter Modifier

Azure Meetup TORINO

alveo EXPERTISE

TD SYNNEX

#GlobalAzureTorino

Filter and Filter Context Definition



CALCULATE (<Scalar Expression>, [Filter/GlobalModifier 1], [Filter/GlobalModifier 2], …)

A Filter is a set of tuples for one or more columns.

A set of Filters is called Filter Context.

CALCULATE Filters can be expressed, **in some circumstances**, as Predicates.

Example: CALCULATE ([Sales], Products[Color] = "Red")

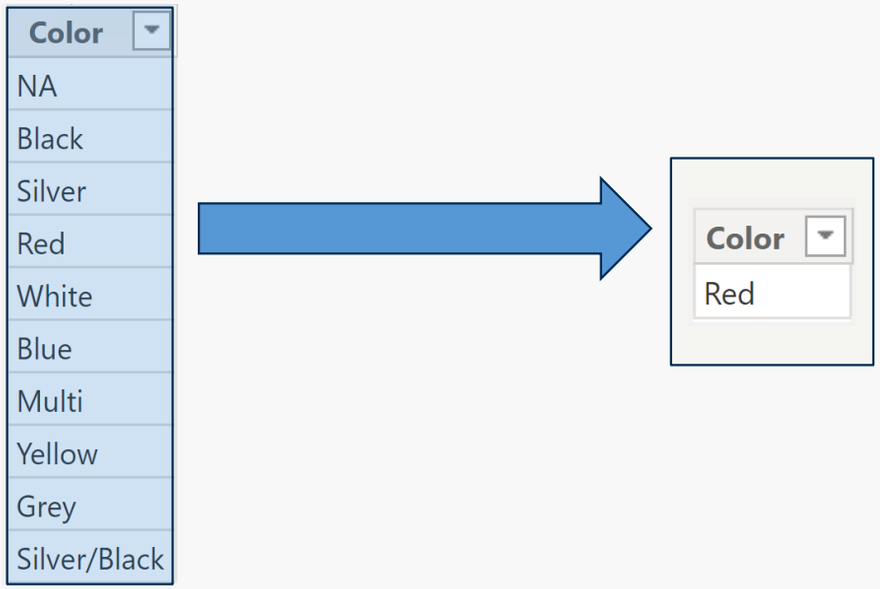
Creating Filters is the way to change the Filter Context through CALCULATE.

#GlobalAzureTorino

More on Filters

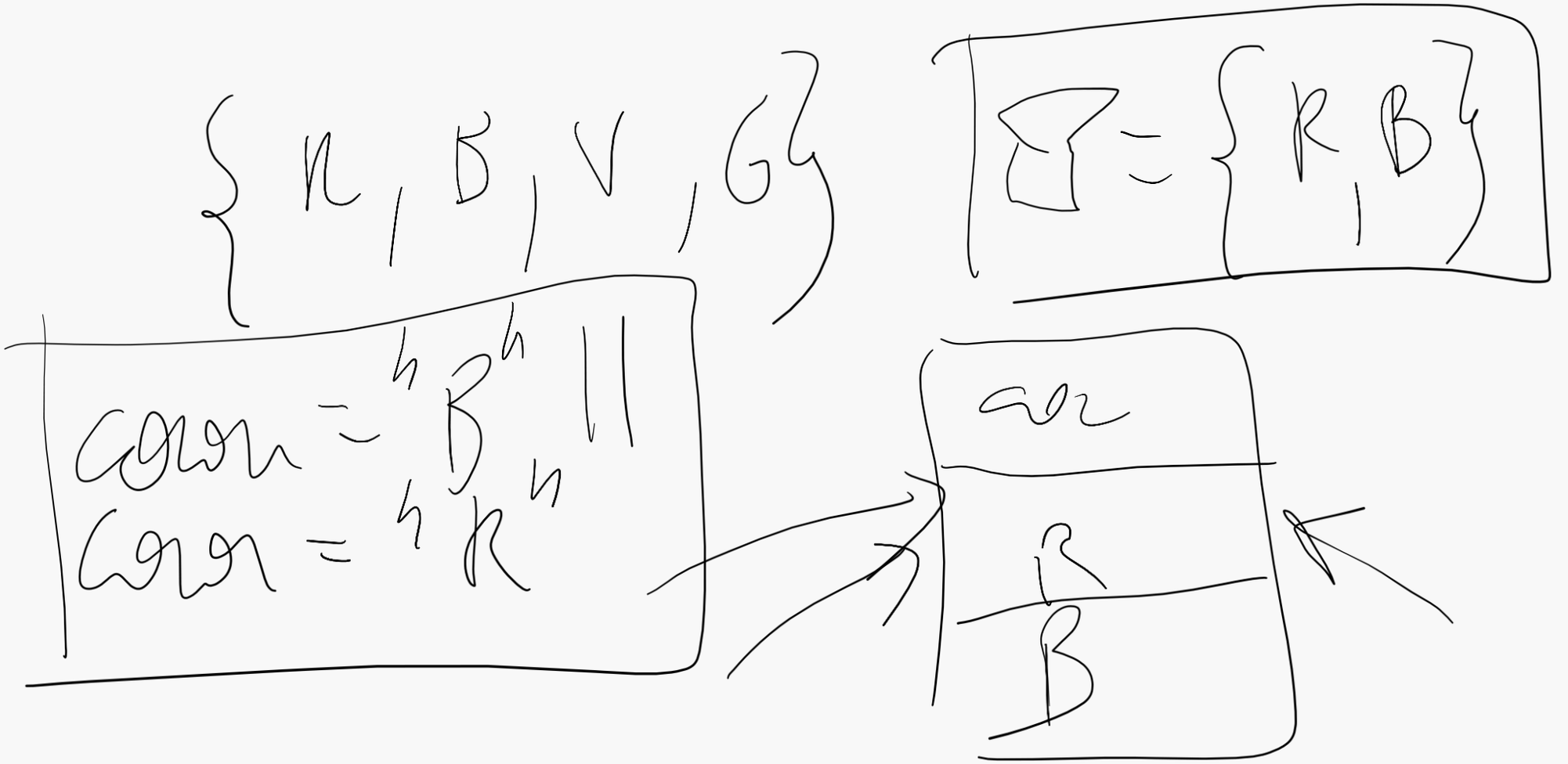
CALCULATE ([Sales], Product[Color] = "Red") is translated into:

CALCULATE ([Sales], FILTER (ALL (Product[Color]), Product[Color] = "Red"))



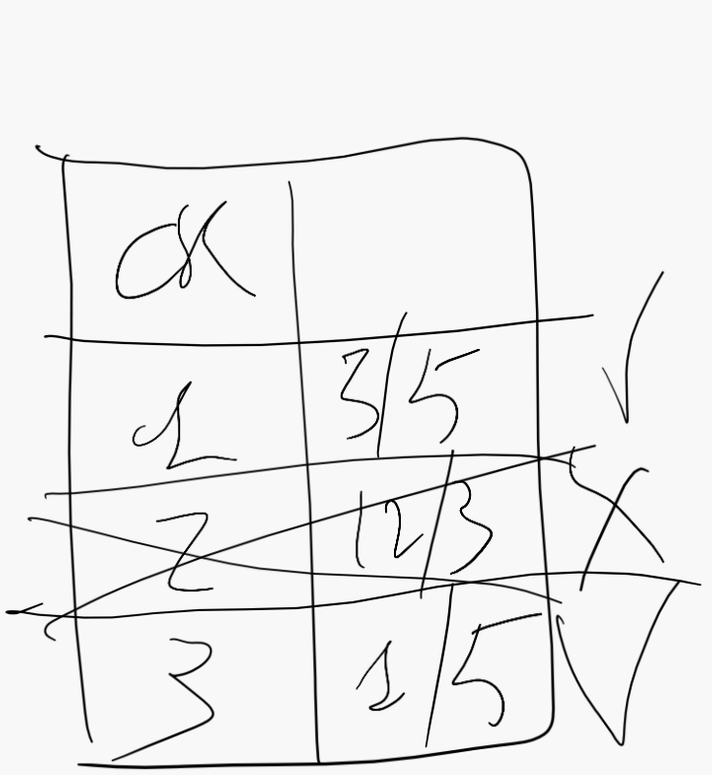
#GlobalAzureTorino

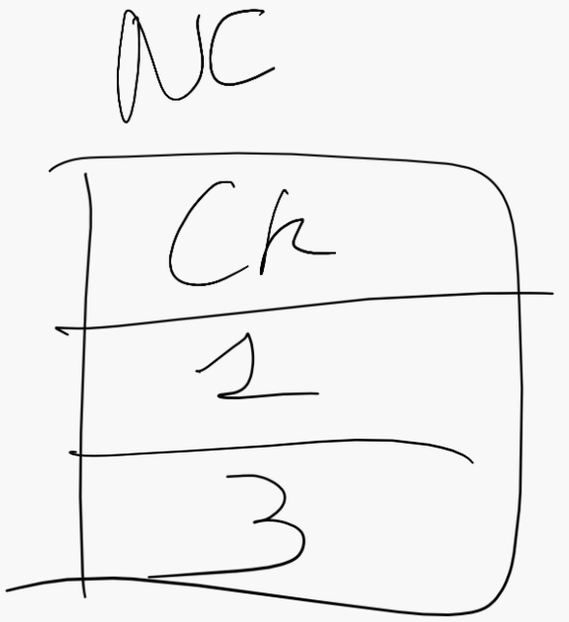
Filter and Filter Context Definition

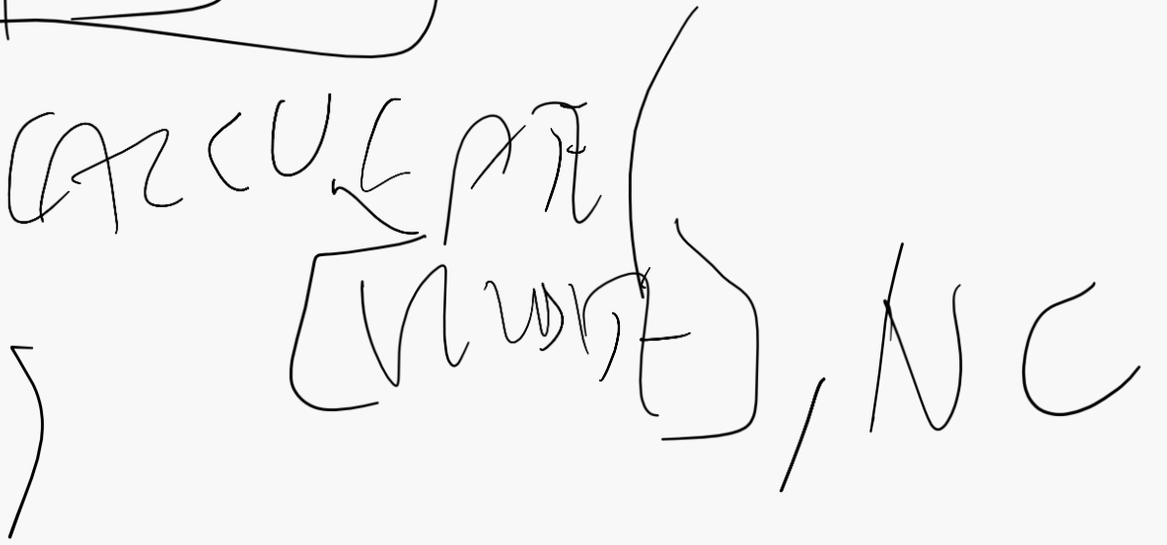


#GlobalAzureTorino

Filter and Filter Context Definition (Application)







#GlobalAzureTorino

More on Filters



CALCULATE (<Scalar Expression>, [Filter/GlobalModifier 1], [Filter/GlobalModifier 2], …)

CALCULATE Filters, though, are Tables, not Predicates.

The syntax sugar of a predicate can be used when filtering a set of columns from the same table for a set of specific values, that can also be expressed as a DAX expression (NOT an explicit call to a measure, though, more on this later).

Examples ALLOWED:
CALCULATE ([Sales], Product[Color] = "Red")
CALCULATE ([Sales], Product[Color] = "Red", Product[Size] = "S")
CALCULATE ([Sales], Product[Color] = "Red" && Product[Size] = "S")
CALCULATE ([Sales], Product[Color] = "Red" || Product[Size] = "S")
CALCULATE ([Sales], Product[ListPrice] = MAX (Product[ListPrice])

#GlobalAzureTorino

More on Filters



CALCULATE (<Scalar Expression>, [Filter/GlobalModifier 1], [Filter/GlobalModifier 2], …)

CALCULATE Filters, though, are Tables, not Predicates.

This means it is utterly important to be expert of DAX table functions like

SUMMARIZE
CROSSJOIN
GENERATE
VALUES
DISTINCT
FILTER
ALL XXX

To create the Filters you need !

#GlobalAzureTorino

More on Filters



CALCULATE (<Scalar Expression>, [Filter/GlobalModifier 1], [Filter/GlobalModifier 2], …)

CALCULATE Filters, though, are Tables, not Predicates.

The syntax sugar of a predicate cannot be used when filtering a set of columns from different tables for a set of specific values and when the values are to be calculated through a measure.

Examples NOT ALLOWED:

1 CALCULATE ([Sales], Product[Color] = [Top Sales Color])
2 CALCULATE ([Sales], Product[Color] = "Red" || Customer[EnglishEduction] = "Bachelors")
3 CALCULATE ([Sales], Product[Color] = "Red" && Customer[EnglishEduction] = "Bachelors")

note: the 3° example can be solved simply by
CALCULATE ([Sales], Product[Color] = "Red", Customer[EnglishEducation] = "Bachelors")

#GlobalAzureTorino

Filter Modifier



CALCULATE (<Scalar Expression>, [Filter/GlobalModifier 1], [Filter/GlobalModifier 2], …)

Each Filter can be applied with or without a Filter Modifier called KEEPFILTERS which, if used, changes the semantics of the Filter injection from the default OVERRIDE to INTERSECT. A Filter Modifer is not a Global Modifier as it affects only the semantics of a specific Filter.

#GlobalAzureTorino

Topic 3 Global Modifiers and Algorithm

CALCULATE Context Transition and Global Modifiers
CALCULATE Algorithm
CALCULATE Algorithm re-cap examples (Demo)

Azure Meetup TORINO

alveo EXPERTISE

TD SYNNEX

#GlobalAzureTorino

Context Transition

Context Transition in a Calculated Column:

OVERWRITE / INTERSECT is irrelevant as the Filter Context is initially empty

Filter Context after Context Transition

```
1 Product Sales Column =
2 [Sales]
```

PK	EPN	COLOR	SSL
313	Road...	Red	100

LP	S	CLASS
3578,27	52	H

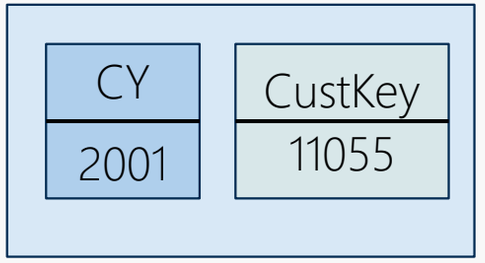
ProductKey	EnglishProductName	Color	SafetyStockLevel	ListPrice	Size	Class	Product Sales Column
312	Road-150 Red, 48	Red	100	3578,27	48	H	1.205.877
310	Road-150 Red, 62	Red	100	3578,27	62	H	1.202.299
313	Road-150 Red, 52	Red	100	3578,27	52	H	1.080.638
314	Road-150 Red, 56	Red	100	3578,27	56	H	1.055.590
							1.005.494
							979.961
							979.036
							961.601
							954.716

The Filter Context is initially empty (this is a calculated Column) and a Row Context is active on he Product Table. The implicit CALCULATE of a measure reference triggers Context Transition. The Filter Context is then filled with Filters and the Row Context is invalidated. In case of multiple row contexts, this will happen for all of them. The inner Row Context will prevail on the outer.

#GlobalAzureTorino

Context Transition

```
1  Sales Cust Over 1.000 =
2  SUMX(
3      VALUES( Customer[CustomerKey] ),
4      VAR SalesCurrentCustomerCurrentContext = [Sales]
5      RETURN
6      IF (
7          SalesCurrentCustomerCurrentContext > 1000,
8          SalesCurrentCustomerCurrentContext
9      )
10 )
```



Customerkey	[Sales]
11000	34.000
11055	68.000
12078	45.000
13456	57.000
13689	13.000

Context Transition in a Measure: here the *Sales Cust Over 1.000* Measure will only consider Customers that, in the current selection, have Sales above 1.000. OVERWRITE is again irrelevant as there is no Filter, initally, on the Customer table in the Filter Context.

CalendarYear	Sales	Sales Cust Over 1.000
2001	3.266.374	3.187.376
2002	6.530.344	6.228.613
2003	9.791.060	8.727.456
2004	9.770.900	8.321.146
Total	**29.358.677**	**27.847.299**

#GlobalAzureTorino

Context Transition

```
1 Monthly Average Sales =
2 AVERAGEX (
3     KEEPFILTERS( VALUES ( 'Calendar'[MonthNumberOfYear] ) ),
4     [Sales]
5 )
```

CalendarYear	Sales	Monthly Average Sales
2003	**2.928.769**	**1.464.384**
11	1.196.981	1.196.981
12	1.731.788	1.731.788
2004	**2.802.725**	**1.401.362**
1	1.340.245	1.340.245
2	1.462.480	1.462.480
Totale	**5.731.494**	**1.432.873**



Filter Context in which [Monthly Average Sales] is executed

Year	Month
2003	11
2003	12
2004	1
2004	2

Average = 1.432.873

Month	Sales
11	**1.196.981**
12	**1.731.788**
1	**1.340.245**
2	**1.462.480**

Filter Context in which [Sales], due to the implicit CALCULATE and the Context Transition, is executed, Row by row on VALUES (Calendar[Month NumberOfYear])

#GlobalAzureTorino

Global Modifiers

- REMOVEFILTERS ( ) - Alias for ALL ( )
- ALLXXX ( ) - Filter Removal with several different semantics
- USERELATIONSHIP ( ) - Modify the Columns involved in the model Relationships
- CROSSFILTER ( ) - Modify the Cross-Filter direction in the model Relationships

#GlobalAzureTorino

CALCULATE Algorithm



1 – Evaluate the explicit Filters, if any, **in the Filter Context active at the time of the call to CALCULATE** and create a copy of this Filter Context;

2 – Perform Context Transition on **all row contexts (RC) active at the time of the CALCULATE** call (the Filter Context starts to change, inner RC will prevail on outer);

3 – Apply the Global Modifiers, if any (further Filter Context change);

4 – Apply the explicit Filters, evaluated in step 1, if any (final Filter Context change), each **with our without the Filter Modifier KEEPFILTERS**;

5 – Evaluate the scalar expression in the modified Filter Context, return the result, then put the Filter Context active at the time of the call to CALCULATE back in force.

#GlobalAzureTorino

CALCULATE Algorithm



The algorithm steps are performed in the specific order outlined, therefore the two following expressions give the same result:

CALCULATE ([Revenues], Products[Color] = "Red", REMOVEFILTERS (Products))

CALCULATE ([Revenues], REMOVEFILTERS (Products), Products[Color] = "Red")

In general, the order in which you insert Filters and Global Modifiers is irrelevant. What IS relevant is managing the calls to functions to take advantage of the algorithm

#GlobalAzureTorino

CALCULATE Algorithm



Focus on the details of the algorithm that are crucial to get the result we are looking for:

1 – Evaluate the explicit Filters, if any, **in the Filter Context active at the time of the call to CALCULATE** and create a copy of this Filter Context

The bold part is extremely important: Filters are a MEMORY to what the Filter Context was at the time of the CALCULATE call, so you can restore a part of it if you need to

#GlobalAzureTorino

CALCULATE Algorithm

Filters are a MEMORY to what the Filter Context was at the time of the CALCULATE call, so you can restore a part of it if you need to. Example:

```
1 Sales of the Year =
2 CALCULATE(
3     [Sales],
4     REMOVEFILTERS ( 'Calendar' ),
5     VALUES ( 'Calendar'[CalendarYear] )
6 )
```

CalendarYear EnglishMonth Name	2001 Sales	Sales of the Year	2002 Sales	Sales of the Year	2003 Sales	Sales of the Year	2004 Sales	Sales of the Year	**Total Sales**	**Sales of the Year**
January		3.266.374	596.747	6.530.344	438.865	9.791.060	1.340.245	9.770.900	**2.375.857**	**29.358.677**
February		3.266.374	550.817	6.530.344	489.090	9.791.060	1.462.480	9.770.900	**2.502.387**	**29.358.677**
March		3.266.374	644.135	6.530.344	485.575	9.791.060	1.480.905	9.770.900	**2.610.615**	**29.358.677**
April		3.266.374	663.692	6.530.344	506.399	9.791.060	1.608.751	9.770.900	**2.778.842**	**29.358.677**
May		3.266.374	673.556	6.530.344	562.773	9.791.060	1.878.318	9.770.900	**3.114.646**	**29.358.677**
June		3.266.374	676.764	6.530.344	554.799	9.791.060	1.949.361	9.770.900	**3.180.924**	**29.358.677**
July	473.388	3.266.374	500.365	6.530.344	886.669	9.791.060	50.841	9.770.900	**1.911.263**	**29.358.677**
August	506.192	3.266.374	546.001	6.530.344	847.414	9.791.060		9.770.900	**1.899.607**	**29.358.677**
September	473.943	3.266.374	350.467	6.530.344	1.010.258	9.791.060		9.770.900	**1.834.668**	**29.358.677**
October	513.329	3.266.374	415.390	6.530.344	1.080.450	9.791.060		9.770.900	**2.009.169**	**29.358.677**
November	543.993	3.266.374	335.095	6.530.344	1.196.981	9.791.060		9.770.900	**2.076.070**	**29.358.677**
December	755.528	3.266.374	577.314	6.530.344	1.731.788	9.791.060		9.770.900	**3.064.630**	**29.358.677**
Total	**3.266.374**	**3.266.374**	**6.530.344**	**6.530.344**	**9.791.060**	**9.791.060**	**9.770.900**	**9.770.900**	**29.358.677**	**29.358.677**

This Filter is evaluated in the Filter Context active at the time of the CALCULATE call, so before REMOVEFILTERS is called. Therefore, even though, after REMOVEFILTERS, there is no trace of the [CalendarYear] value, we can restore it!

#GlobalAzureTorino

CALCULATE Algorithm



Focus on the details of the algorithm that are crucial to get the result we are looking for:

2 – Perform Context Transition on **all row contexts (RC) active at the time of the CALCULATE** call (the Filter Context starts to change, inner RC will prevail on outer)

The bold part is again extremely important: If more than one (nested) row context was active at the time of the CALCULATE call, all of them will be converted to an equivalent Filter Context. Inner row contexts will prevail on outer

#GlobalAzureTorino

CALCULATE Algorithm

If more than one (nested) row context was active at the time of the CALCULATE call, all of them will be converted to an equivalent Filter Context. Inner row contexts will prevail on outer.

Example:

```
1 Nested Row Context Product Sales =
2 -- Outer Row Context on Table Product
3 SUMX(
4     -- Inner Row Context on Table Product
5     'Product',
6     [Sales]
7 )
```

A set of filters will be added to the initially empty Filter Context, one for each column in the outer Row Context

Here the inner Row Context will go again one row at a time scanning the full table, and the iterated row will prevail on the one iterated from the outer Row Context

CALCULATE will trigger Context Transition and invalidate both Row Contexts

EnglishProductName	Color	SafetyStockLevel	ListPrice	Size	Class	Product Sales Column	Nested Row Context Product Sales
Road-150 Red, 48	Red	100	3578,27	48	H	1.205.877	29.358.677
Road-150 Red, 62	Red	100	3578,27	62	H	1.202.299	29.358.677
Road-150 Red, 52	Red	100	3578,27	52	H	1.080.638	29.358.677
Road-150 Red, 56	Red	100	3578,27	56	H	1.055.590	29.358.677
Road-150 Red, 44	Red	100	3578,27	44	H	1.005.494	29.358.677

#GlobalAzureTorino

CALCULATE Algorithm



Focus on the details of the algorithm that are crucial to get the result we are looking for:

3 – Apply the Global Modifiers, if any (further Filter Context change)

The important detail here is that this step (step 3) comes after Context Transition (step 2): Global Modifers can, therefore, override Context Transition

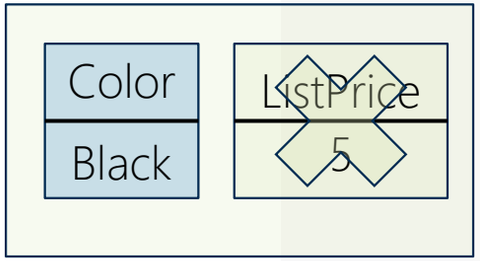
#GlobalAzureTorino

CALCULATE Algorithm

The important detail here is that this step (step 3) comes after Context Transition (step 2): Global Modifers can, therefore, override Context Transition. Example:

```
1 Sales MaxListPrice Products =
2 CALCULATE(
3     [Sales],
4     FILTER(
5         ALL ( 'Product'[ListPrice] ),
6         'Product'[ListPrice] =
7         CALCULATE( [Max List Price], REMOVEFILTERS( 'Product'[ListPrice] )
8         )
9     )
10 )
```

Color	Sales	Sales MaxListPrice Products
Black	8.838.412	712.123
Blue	2.279.096	1.511.500
Multi	106.471	
NA	435.117	
Red	7.724.331	5.549.897
Silver	5.113.389	628.998
White	5.106	
Yellow	4.856.756	1.480.507
Totale	**29.358.677**	**5.549.897**



Max List Price = MAX ('Product'[ListPrice])

ALL (Product[ListPrice])	[MaxListPrice]	CALCULATE ([MaxListPrice], REMOVEFILTERS (Product[Listprice]))
5	5	15
7	7	15
9	9	15
12	12	15
15	15	15

#GlobalAzureTorino

CALCULATE Algorithm



The important detail here is that this step (step 3) comes after Context Transition (step 2): Global Modifers can, therefore, override Context Transition. Example with better DAX code, no Context Transition at all!:

```
1 Sales MaxListPrice Products IMPROVED DAX =
2 VAR _MaxListPrice = [Max List Price]
3 RETURN
4 CALCULATE(
5     [Sales],
6     'Product'[ListPrice] = _MaxListPrice
7 )
```

Color	Sales	Sales MaxListPrice Products	Sales MaxListPrice Products IMPROVED DAX
Black	8.838.412	712.123	712.123
Blue	2.279.096	1.511.500	1.511.500
Multi	106.471		
NA	435.117		
Red	7.724.331	5.549.897	5.549.897
Silver	5.113.389	628.998	628.998
White	5.106		
Yellow	4.856.756	1.480.507	1.480.507
Totale	**29.358.677**	**5.549.897**	**5.549.897**

Remember that Variables in DAX are… constant values!

#GlobalAzureTorino

CALCULATE Algorithm



Focus on the details of the algorithm that are crucial to get the result we are looking for:

4 – Apply the explicit Filters, evaluated in step 1 if any, to the Filter Context (final change), **with our without the Filter Modifier KEEPFILTERS**

The bold part is again extremely important: KEEPFILTERS is applied to a Filter, so it is always applied after Global Modifiers and you are free to decide, on each Filter, if you want or not KEEPFILTERS (INTERSECT)

#GlobalAzureTorino

CALCULATE Algorithm

```
1 Trendy Color Sales =
2 CALCULATE(
3     [Sales],
4     'Product'[Color] IN {"Black", "Blue", "Silver"}
5 )
```

SalesTerritoryCountry	Sales	Trendy Color Sales
Australia	4.697.632	4.916.822
Canada	1.169.062	978.286
France	1.298.137	1.534.561
Germany	1.476.614	1.684.235
United Kingdom	1.681.648	2.031.972
United States	5.083.784	5.085.021
Total	**15.406.876**	**16.230.897**



How can Trendy Color Sales be higher than Sales? This is due to OVERWRITE. The existing Filter on the Color (slicer) will be removed before injecting the CALCULATE one and, therefore, always and only the three trendy colors will be considered

#GlobalAzureTorino

CALCULATE Algorithm

When and why is KEEPFILTERS useful (continued)?

```
1 Trendy Color Sales =
2 CALCULATE(
3     [Sales],
4     KEEPFILTERS(
5         'Product'[Color] IN {"Black", "Blue", "Silver"}
6     )
7 )
```

SalesTerritoryCountry	Sales	Trendy Color Sales
Australia	4.697.632	553.453
Canada	1.169.062	169.503
France	1.298.137	188.680
Germany	1.476.614	266.537
United Kingdom	1.681.648	321.907
United States	5.083.784	779.015
Total	**15.406.876**	**2.279.096**



Using KEEPFILTERS, the existing Filter on the Color (slicer) will be kept and then intersected with the Filter injected by CALCULATE

Note: you can decide on each Filter whether or not to use KEEPFILTERS but, before you get to a rule like «I always insert it», please think about it as sometimes you do not want it!

#GlobalAzureTorino

francesco.bergamaschi@kubisco.com (business)

francesco.bergamaschi@unibo.it (students)

https://it.linkedin.com/in/francescobergamaschi

kubisco www.kubisco.com

POWER BIUG Italy Power BI User Group


BY COMMUNITY • FOR COMMUNITY •
#GLOBALAZURE
TORINO